

Lecture 11

Monday March 3 2026

Quiz 2 will be due **March 13th**

Midterm will be in class **March 26th**

Project 1 will be due **April 2nd**

Recap

- Last lectures were on CPU Scheduling and the different algorithms to provide fairness
- Preemptive and Non-preemptive
- Shortest time, Round Robin, Priority
- The exercise on slide 51 will be on the quiz make sure to work through it

Synchronization Background

- Concurrent access to shared data may result in data inconsistency
- maintaining data consistency requires mechanisms to ensure orderly execution of cooperating processes

Race Condition

- **Race condition:** the situation where several processes access and manipulate shared data concurrently. the final value of the shared data depends upon which process finishes last
- To prevent race conditions, concurrent processes must be **synchronized**
 - ensure that only one process at a time is manipulating the variable counter
 - The statements
 - `count++;`
 - `count--;`must be performed atomically
- **atomic operation** means an operation without interruption

I/O and variable sharing

- Significant race conditions exist in I/O and variable sharing
 - In this case, replacing the global variables with local variables did not solve the problem the the local variables are still racing to use the I/O even if they are not overwriting each other

- lucky scheduling is when the scheduler just so happens to allow the full running of a block of code without interruption
- unlucky scheduling is when the scheduler just so happens to allow the full running of a block of code without interruption
- Generally problematic race conditions may occur whenever resources and/or data are shared (by processes unaware of each other) called **critical sections (CS)** or **critical region (CR)**

Critical Sections/Regions CS/CR

- segment of code in which the process may be changing shared data
- The "indivisible" execution blocks are critical regions
 - a section of code that should be executed by only one process or thread at a time
- Although it is not necessarily the same region of memory or section of program in both processes
- these regions cannot be interleaved or executed in parallel (pseudo or real)

Solution

- Mutual Exclusion - only one process can be in a critical section at one moment in time
- Progress - you cannot have deadlocks preventing other processes from accessing the CS
- Bounded waiting - a bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.
 - assume that each process executes at a nonzero speed
 - no assumption concerning relative speed of the **N** processes

Mutual Exclusion

- CR can be protected from concurrent access by padding them with entrance and exit gates:
 - a thread must wait until the lock is removed
- Implementation 1 - Disable the hardware interrupts
 - Works but unreasonable
 - What are the issues?
 - creates problems with the entire OS which cannot do any work in this case
 - If needed it should only be done with 2-3 lines of code to avoid problems
 - forgetting to enable the hardware interrupt will cause major problems
- Implementation 2 - simple lock variable
 - lock is a shared variable

- entering the CR means testing and then setting the lock
- exiting means resetting the lock
- run into problems when multiple processes reach the unlocked variable at the same time and both try to set the lock
- Implementation 3 - "indivisible" lock variable
 - The indivisibility of the "test-lock-and-set-lock" operation can be implemented with the hardware level (assemble
 - Test and Set (**TSL**)
 - Spin lock (**SL**)